

Learn Home Automation with Arduino

Lesson 6: Internet Connectivity with ESP8266

Internet Connectivity with ESP8266 – Study Notes

Key Concepts

1. **ESP 01 (ESP8266) Wi-Fi module basics** – pinout, power requirements, AT command mode vs. Arduino core.
2. **Connecting to a Wi-Fi network** – SSID/password handling, `WiFi.begin()`, connection status checks.
3. **HTTP communication** – forming GET/POST requests, using `WiFiClient` or `HTTPClient` to send sensor data to a cloud endpoint.
4. **Data formatting** – JSON payloads, URL encoded query strings, and why lightweight formats matter for microcontrollers.
5. **Power saving & reliability** – resetting the module, handling reconnections, and basic watchdog concepts.

Summary

The ESP 01 is a low cost Wi-Fi radio based on the ESP8266 chip. When paired with a simple microcontroller (or programmed directly with the Arduino core), it can join a wireless LAN and exchange data with internet services. In this lesson you learn how to power the module safely, configure it to connect to an existing Wi-Fi network, and transmit sensor readings (e.g., temperature, humidity, light level) to a cloud service using HTTP POST requests.

The workflow is straightforward: initialize the serial interface, start the Wi-Fi library, wait for a successful connection, then repeatedly read the sensor, build a JSON payload, and send it to the cloud endpoint. Proper error handling—checking return codes, retrying failed connections, and resetting the ESP 01 when needed—makes the system robust enough for real world deployments.

Important Points

- **Power supply:** ESP 01 draws up to 250 mA during Wi-Fi bursts; use a stable 3.3 V regulator and decoupling capacitors (100 µF).
- **GPIO0 & GPIO2:** Must be HIGH for normal operation; pulling GPIO0 LOW puts the chip into flashing mode.
- **Arduino core vs. AT firmware:** The Arduino core lets you program the ESP directly (recommended for beginners).
- **Wi-Fi connection code pattern:**

```
```cpp
```

```
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
```

```

 delay(500);
 Serial.print(".");
}
Serial.println("\nConnected, IP: " + WiFi.localIP().toString());
...

```

- **HTTP POST with JSON:**

```

...cpp
HTTPClient http;
http.begin("http://api.example.com/sensor");
http.addHeader("Content-Type", "application/json");
int httpResponseCode = http.POST(jsonPayload);
...

```

- **JSON payload example:** `{"temp":23.5,"hum":58}`

- **Reconnection strategy:** If `WiFi.status()` becomes `WL_DISCONNECTED`, call `WiFi.reconnect()` and optionally reset the module after several failed attempts.

- **Security note:** ESP 01 does not support TLS out of the box in the Arduino core; for production use consider ESP 01S with SSL libraries or a proxy that accepts plain HTTP.

---

## Examples

### Example 1 – Send Temperature to ThingSpeak

```

...cpp
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
const char* ssid = "YourSSID";
const char* password = "YourPASS";
const char* apiKey = "YOUR_THINGSPEAK_WRITE_API_KEY";
void setup() {
 Serial.begin(115200);
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
 }
 Serial.println("\nWi-Fi connected");
}
void loop() {
 float temperature = analogRead(A0) * (3.3 / 1023.0) * 100; // dummy conversion
 String url = String("http://api.thingspeak.com/update?api_key=") + apiKey +
 "&field1=" + String(temperature, 2);
 HTTPClient http;
 http.begin(url);

```

```

int httpCode = http.GET(); // GET request is enough for ThingSpeak
Serial.println("HTTP code: " + String(httpCode));
http.end();
delay(20000); // ThingSpeak limit: 15 /s per update
}
```

```

Explanation: The sketch connects to Wi Fi, reads a mock temperature sensor, builds a URL with query parameters, and sends a GET request to ThingSpeak's public API. The delay respects the service's rate limit.

Example 2 – POST JSON to a Custom Cloud Endpoint

```

```cpp
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <ArduinoJson.h> // install via Library Manager
const char* ssid = "HomeNetwork";
const char* password = "home1234";
void setup() {
 Serial.begin(115200);
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
 }
 Serial.println("\nConnected");
}
void loop() {
 // Simulated sensor values
 float humidity = random(30, 70);
 float pressure = random(990, 1025);
 // Build JSON payload
 StaticJsonDocument<200> doc;
 doc["humidity"] = humidity;
 doc["pressure"] = pressure;
 String payload;
 serializeJson(doc, payload);
 // Send POST request
 HTTPClient http;
 http.begin("http://myserver.com/api/sensor"); // change to your URL
 http.addHeader("Content-Type", "application/json");
 int httpResponse = http.POST(payload);
 Serial.printf("POST response: %d\n", httpResponse);
 http.end();
}
```

```

```
delay(10000); // send every 10 /s
}
...
```

Explanation: This sketch demonstrates creating a JSON object with the **ArduinoJson** library, then posting it to a user defined endpoint. The ``Content-Type: application/json`` header tells the server how to parse the body.

Quick Review

1. **What voltage and current considerations must be addressed when powering an ESP 01?**
2. **Which Arduino function checks the current Wi Fi connection status?**
3. **How do you format a simple JSON payload containing temperature and humidity?**
4. **Why might you prefer a POST request over a GET request for sending sensor data?**
5. **Name one method to improve reliability when the Wi Fi connection drops.**

Further Reading

- **ESP8266 Arduino Core Documentation** – deeper dive into Wi Fi, OTA updates, and deep sleep modes.
- **Secure Communication** – using ``WiFiClientSecure`` for HTTPS, certificate handling, and alternatives like MQTT over TLS.
- **MQTT Protocol** – lightweight publish/subscribe model ideal for IoT sensor networks.
- **Power Management** – using the ESP8266's deep sleep feature to extend battery life.
- **Cloud Platforms** – comparison of ThingSpeak, Adafruit IO, Blynk, and AWS IoT for beginner projects.

